# MakeNoise<MapDomain<AtomDomain<TK>, AtomDomain<TV>>, LOPInfDistance<P, AbsoluteDistance<QI>>, MO> for FloatExpFamily<P>

Michael Shoemate

Proves soundness of the implementation of `MakeNoiseThreshold` over hash maps for `FloatExpFamily` in `mod.rs` at commit f5bb719 (outdated[1]).

This mechanism samples from the `FloatExpFamily` distribution, where the tails saturate to the positive and negative infinity floats in the native float type. This is done by rounding floats to a fractional grid, adding noise from from the `ZExpFamily` distribution (supported on all integers) to the numerator of the fraction (with noise scale multiplied by the denominator), and then converting back to the nearest float, or saturating to positive or negative infinity.

## 1 Hoare Triple

### Precondition

#### Compiler-Verified

- Generic `TK` implements trait `Hashable`
- Generic `TV` implements trait `Float`
- Const-generic `P` is of type `usize`
- Generic `QI` implements trait `Number`
- Generic `MO` implements trait `Measure`
- Type `i32` implements trait `ExactIntCast<T as FloatBits>::Bits>`, This requirement means that the raw bits of `T` can be exactly cast to an `i32`.
- Type `RBig` implements traits `TryFrom<T>` and `TryFrom<QI>`. This is for fallible exact casting from input sensitivity to a rational in the privacy map.
- Type `ZExpFamily<P>` implements traits `NoiseThresholdPrivacyMap<LOPInfDistance<P, AbsoluteDistance<RBig>>, MO>` This bound requires that it must be possible to construct a privacy map for this combination of noise distribution, distance type and privacy measure.

#### User-Verified

None

[1] See new changes with `git diff f5bb719..c6ec4181 rust/src/measurements/noise_threshold/nature/float/mod.rs`

## Pseudocode

```
1  class FloatExpFamily:
2      def make_noise_threshold(
3          self,
4          input_space: tuple[
5              MapDomain[AtomDomain[TK], MapDomain[TV]], L0PInfDistance[P, AbsoluteDistance[QI
       ]]
6          ],
7          threshold: TV,
8      ) -> Measurement[
9          MapDomain[AtomDomain[TK], AtomDomain[TV]],
10         HashMap[TK, TV],
11         L0PInfDistance[P, AbsoluteDistance[QI]],
12         MO,
13     ]:
14         scale, k = self.scale, self.k
15         distribution = ZExpFamily(
16             scale=integerize_scale(scale, k)
17         )  #
18
19         if threshold.is_sign_negative():
20             raise f"threshold ({threshold}) must not be negative"
21
22         r_threshold = RBig.try_from(threshold)
23         r_threshold = x_mul_2k(r_threshold, -k).round()
24
25         t_int = make_float_to_bigint_threshold(input_space, threshold, k)
26         m_noise = distribution.make_noise_threshold(t_int.output_space(), r_threshold)
27         return t_int >> m_noise >> then_deintegerize_hashmap(k)
```

## Postcondition

**Theorem 1.1.** For every setting of the input parameters (`self, input_space, MO, TK, TV, P, QI`) to `make_noise` such that the given preconditions hold, `make_noise` raises an error (at compile time or run time) or returns a valid measurement. A valid measurement has the following properties:

1. (Data-independent runtime errors). For every pair of members $x$ and $x'$ in `input_domain`, `invoke`$(x)$ and `invoke`$(x')$ either both return the same error or neither return an error.

2. (Privacy guarantee). For every pair of members $x$ and $x'$ in `input_domain` and for every pair (`d_in`, `d_out`), where `d_in` has the associated type for `input_metric` and `d_out` has the associated type for
`output_measure`, if $x, x'$ are `d_in`-close under `input_metric`, `privacy_map`(`d_in`) does not raise an error, and `privacy_map`(`d_in`) = `d_out`, then `function`$(x)$, `function`$(x')$ are `d_out`-close under `output_measure`.

*Proof.* Line 17 constructs a new random variable following a distribution equivalent to `FloatExpFamily`, but without tails that saturate to infinity, and with gaps between adjacent point on the grid adjusted to one.

Neither constructor `make_float_to_bigint_threshold` nor `MakeNoiseThreshold.make_noise_threshold` have manual preconditions, and the postconditions guarantee a valid transformation and valid measurement, respectively. `then_deintegerize_hashmap` also does not have preconditions, and its postcondition guarantees that it returns a valid postprocessor.

The chain of a valid transformation, valid measurement and valid postprocessor is a valid measurement. □